

Sensei Architectural Pre-Review Report

Implementation charter

1. Definition

The **Sensei Architectural Pre-Review Report** is a deterministic, evidence-linked report generated for a proposed repository change before a human begins detailed review.

It answers:

```
What changed?  
What architectural meaning does the change carry?  
Which governed rules apply?  
Was the actor authorized?  
Did the observed patch remain inside its admitted scope?  
Which architectural behaviors are proven?  
Which evidence is missing, stale, or conflicting?  
How did the result architecture change?  
What still requires human judgment?
```

The report does not replace code review.

It removes mechanical reconstruction, repository archaeology, and already-provable questions from human review so reviewers can focus on the decisions only humans may legitimately make.

Product promise:

Before a human reviews the code, Sensei reviews what the change means.

2. Core architectural law

The report is a **projection**, not a source of authority.

```
PreReviewReport  
!= architecture fact  
!= admission decision  
!= proof discharge
```

```
!= certification
!= completion
```

The report may reference authoritative receipts.

It may never create or upgrade them.

For example:

```
CertificationReceipt says certified
→ report may display certified

report renderer says certified
→ cannot create CertificationReceipt
```

The report must not contain a hidden certification engine.

3. Desired user experience

Local command

```
sensei pre-review
  --base origin/main
  --head HEAD
  --format markdown
  --output sensei-pre-review.md
```

For a governed task:

```
sensei pre-review
  --task-dir .sensei/tasks/task.<id>
  --format markdown
```

Machine-readable output:

```
sensei pre-review
  --task-dir .sensei/tasks/task.<id>
  --format json
```

CI use

```
- uses: globulario/sensei-action@v1
  with:
    mode: pre-review
```

The action may publish:

- a GitHub job summary;
- a stable PR comment;
- JSON as an artifact;
- SARIF findings for file-level blockers.

The core Sensei binary produces report artifacts.

GitHub posting belongs to the action integration, not the core report package.

4. Report coverage levels

The report must declare what evidence was available.

advisory

Available:

- Git diff;
- graph briefing;
- edit-check findings;
- invariants;
- failure modes;
- forbidden fixes;
- required tests.

Unavailable:

- typed task ledger;
- authority and admission receipts;
- result transition;
- certification.

This is still useful, but cannot claim governed mutation or correctness.

governed

Additionally available:

- task binding;
- actor binding;
- authority resolution;
- Admission v2 decision;
- capability consumption;
- observed change;
- scope verification.

The report may state whether the change was authorized and remained in scope.

It cannot claim correctness.

proof_bound

Additionally available:

- result binding;
- evidence receipts;
- proof discharges;
- certification receipt.

The report may display the independently verified certification verdict.

terminal

Additionally available:

- fresh result graph and artifact receipts;
- valid completion receipt.

The report may display terminal architectural closure.

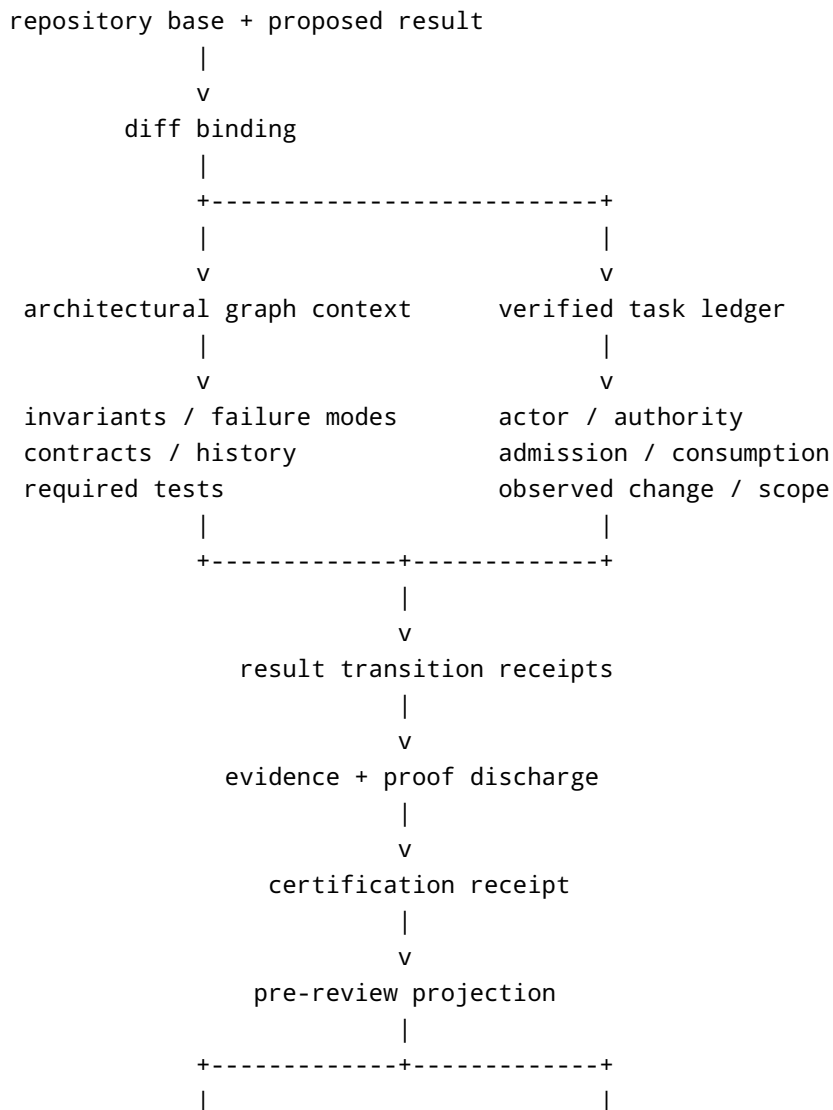
Absence of a later coverage level must not erase useful earlier findings.

5. Non-negotiable laws

1. Every report binds to an exact repository and diff.
2. A task-backed report binds to the verified ledger head.
3. Base and result bindings remain distinct.
4. Mutable projections are never report authority.

5. Findings retain their source and epistemic status.
6. Scope verification does not imply correctness.
7. Certification is displayed only from a verified certification receipt.
8. Completion is displayed only from a verified completion receipt.
9. Missing evidence cannot become PASS.
10. Neural predictions are labeled candidates.
11. Caller-provided booleans cannot establish any verdict.
12. An optional narrative model may summarize facts but cannot create findings.
13. Temporary paths, render time, and PR number do not affect semantic identity.
14. Repeated generation from identical inputs produces identical canonical output.
15. A degraded report must explicitly name what could not be verified.

6. Architectural data flow



v
canonical JSON

v
human Markdown

The report builder consumes typed inputs.

It must not parse terminal text emitted by CLI commands.

7. Report contract

Create a versioned record:

```
type PreReviewReport struct {
    SchemaVersion string
    ReportID      string

    Binding      ReviewBinding
    Coverage     CoverageSummary
    Summary      ExecutiveSummary

    Change      ChangeSummary
    Impact     ArchitecturalImpact
    Governance  GovernanceSummary
    Protection  ProtectionSummary
    History     HistoricalRiskSummary
    Proof       ProofSummary
    Result      ResultArchitectureSummary
    Epistemic   EpistemicSummary

    ReviewerAttention []ReviewerAttentionItem
    Disposition       ReviewDisposition
    Limitations       []string

    ReportDigestSHA256 string
}
```

7.1 Review binding

Must include:

```
binding:
  repository_domain:
  base_revision:
```

```
base_tree_digest_sha256:
head_revision:
head_tree_digest_sha256:
merge_base_revision:
diff_digest_sha256:

task_id:
session_id:
ledger_head_digest_sha256:

base_graph_digest_sha256:
result_graph_digest_sha256:

policy_ids: []
```

PR number, URL, branch names, and repository checkout path are display metadata only.

7.2 Executive summary

Maximum five concise statements:

```
summary:
  purpose:
  architectural_risk:
  current_disposition:
  highest_priority_blocker:
  reviewer_attention_count:
```

These fields must be derived from the structured report.

Do not ask an LLM to invent them from the raw diff.

7.3 Change summary

Include:

- files created;
- files modified;
- files deleted;
- files renamed;
- affected symbols;
- admitted operations;
- observed operations;
- generated artifacts;
- out-of-envelope changes.

7.4 Architectural impact

Include:

- affected components;
- changed boundaries;
- affected contracts;
- authority domains;
- state objects;
- upstream dependents;
- downstream dependencies;
- changed graph relationships;
- architectural risk class.

Every item must include evidence references.

7.5 Governance summary

Include:

```
governance:
  actor:
  verified_roles: []
  authority_status:
  authority_resolution_digest_sha256:
  grant_ids: []
  delegation_ids: []
  selected_mechanisms: []
  admission_status:
  capability_status:
  scope_status:
  violations: []
```

Never derive authorization from briefing text.

Use typed receipts only.

7.6 Protection summary

Include applicable:

- invariants;
- contracts;
- failure modes;
- forbidden fixes;

- required tests;
- governed exceptions;
- intended or desired directions.

For each item:

```
- id:
  title:
  severity:
  applicability:
  status:
  evidence_refs: []
```

7.7 Historical risk

Include:

- related incidents;
- reverted changes;
- previously attempted forbidden fixes;
- recurring failure patterns;
- relevant architectural decisions.

Do not include unrelated repository history merely because keywords match.

7.8 Proof summary

Include:

- required proof obligations;
- required slots;
- discharged slots;
- unresolved slots;
- static evidence;
- test receipts;
- runtime evidence;
- artifact receipts;
- stale receipts;
- conflicted receipts;
- waivers;
- certification verdict.

The report must distinguish:

test executed

from:

test discharges this architectural proof slot

7.9 Result architecture summary

Once Phase 7 exists, include:

- base graph digest;
- result graph digest;
- result-tree digest;
- generated artifact freshness;
- components added or removed;
- boundaries added or removed;
- authority changes;
- contract changes;
- proof-requirement changes;
- newly introduced contradictions;
- invalidated old proofs or certifications.

This section remains unavailable under `advisory` and early `governed` coverage.

7.10 Epistemic summary

Group all load-bearing statements by:

observed
governed
deterministically_inferred
model_candidate
contradicted
unknown
stale
not_applicable
uncertifiable

The report must never merge these into one confidence score.

7.11 Reviewer attention

This is the flagship section.

Each item must contain:

```
type ReviewerAttentionItem struct {  
    ID            string  
    Category      string  
    Question      string  
    WhyItMatters  string  
    Blocking      bool  
    Severity      string  
    EvidenceRefs  []string  
    RelatedFiles  []string  
    AllowedAnswers []string  
    ResolutionPath string  
}
```

Examples:

Is this direct writer a temporary migration exception or a new permanent ownership path?

Does this behavior require runtime evidence, or is static and test proof sufficient under the governed obligation?

Should the compatibility boundary receive an expiring waiver?

Limit the default report to the highest-value three to seven human questions.

Put the complete unresolved set in JSON.

8. Review dispositions

Create a closed vocabulary separate from certification verdicts:

```
ready_for_human_review  
mechanical_repair_required  
governance_required  
architect_decision_required  
evidence_required  
scope_violation  
cannot_verify
```

```
certified
terminally_closed
```

Priority order:

```
cannot_verify
> scope_violation
> governance_required
> mechanical_repair_required
> evidence_required
> architect_decision_required
> ready_for_human_review
> certified
> terminally_closed
```

`certified` requires a valid certification receipt.

`terminally_closed` requires a valid completion receipt.

A report may be `ready_for_human_review` while containing advisory warnings.

9. Reviewer-attention derivation

Implement deterministic ranking.

Candidate attention items come from:

1. load-bearing closure blockers;
2. unresolved contradictions;
3. unknown direction;
4. missing or conflicting authority;
5. scope violations;
6. mandatory proof slots without compatible evidence;
7. required architect questions;
8. active waivers nearing expiry;
9. significant result-graph changes;
10. high-value neural candidates, when enabled.

Suggested rank:

```
attention value =
    blocking weight
    + severity weight
```

- + task relevance
- + architectural reach
- + irreducible human-decision weight
- duplicate penalty
- mechanically-resolvable penalty

Weights must be versioned and deterministic.

Do not use model confidence as the primary rank.

10. Optional narrative generation

An LLM may later generate a prose summary from canonical report JSON.

That narrative must live in a separate field:

```
narrative:
  generated_by:
  model:
  input_report_digest_sha256:
  text:
  authoritative: false
```

Rules:

- the model cannot add findings;
- every referenced finding ID must exist;
- factual statements must map to report fields;
- failure to generate narrative does not fail the report;
- canonical report identity excludes the narrative by default.

The first production version should not require an LLM.

11. Go package design

Create:

```
golang/architecture/prereview/
  model.go
  vocabulary.go
  input.go
```

```
binding.go
collect.go
normalize.go
validate.go
disposition.go
attention.go
impact.go
delta.go
render_json.go
render_yaml.go
render_markdown.go
verify.go
errors.go
```

```
model_test.go
binding_test.go
disposition_test.go
attention_test.go
render_test.go
determinism_test.go
fixtures_test.go
adversarial_test.go
```

The package must not depend on `cmd/awg`.

Use small source interfaces:

```
type DiffSource interface {
    ResolveReviewDiff(ctx context.Context, req DiffRequest) (BoundDiff, error)
}

type GraphSource interface {
    CollectArchitecturalContext(ctx context.Context, req GraphRequest)
    (GraphContext, error)
}

type LedgerSource interface {
    LoadVerifiedTask(ctx context.Context, taskDir string) (TaskContext, error)
}

type ResultSource interface {
    LoadResultTransition(ctx context.Context, taskDir string) (ResultContext,
    error)
}

type CertificationSource interface {
```

```

    LoadCertification(ctx context.Context, taskDir string)
    (CertificationContext, error)
}

type NeuralSource interface {
    LoadCandidates(ctx context.Context, req NeuralRequest) (NeuralContext,
    error)
}

```

This allows testing without Git, gRPC, Oxigraph, or filesystem side effects.

12. Reuse existing Sensei surfaces

Do not build a second architectural evaluator.

Reuse:

- briefing and impact queries;
- edit-check findings;
- gate policy resolution;
- task closure report;
- authority-resolution receipts;
- Admission v2 receipts;
- ledger projections only as caches;
- evidence receipts;
- proof discharges;
- certification receipts;
- result-transition receipts;
- completion receipts;
- neural candidate artifacts.

The existing CLI gate currently contains file-finding and SARIF rendering logic.

Refactor reusable output models gradually rather than copying them into pre-review.

Suggested shared package:

```

golang/architecture/reviewoutput/
    finding.go
    sarif.go
    markdown.go

```

Do not make the first pre-review PR a broad rewrite of the current gate.

13. Schemas and fixtures

Create:

```
docs/design/architectural-pre-review-v1.md

docs/schemas/architectural-pre-review/v1/
  report.schema.json
  reviewer-attention.schema.json

docs/fixtures/architectural-pre-review/v1/
  advisory-clean/
  advisory-blocked/
  governed-scope-verified/
  governance-required/
  evidence-required/
  certified/
  terminally-closed/
  cannot-verify/
  neural-candidates/
  invalid/
```

Invalid fixtures must include:

- caller-forged certification;
- completion without receipt;
- report bound to wrong diff;
- stale ledger head;
- result graph mismatch;
- unresolved finding marked PASS;
- neural candidate rendered as governed fact;
- duplicate reviewer questions;
- temporary path changing semantic digest.

14. CLI implementation

Create:

```
cmd/awg/cmd_pre_review.go
cmd/awg/cmd_pre_review_test.go
```


Command:

```
sensei pre-review
```

Arguments:

```
--repo  
--base  
--head  
--task-dir  
--active  
--format text|markdown|yaml|json  
--output  
--sarif  
--max-reviewer-items  
--include-neural  
--strict
```

Resolution behavior

When `--task-dir` is provided:

- verify ledger;
- bind report to the ledger head;
- load typed records.

When not provided:

- resolve diff;
- attempt to locate one matching active task;
- when exactly one task matches, use it;
- when none match, generate `advisory`;
- when several match, refuse task-backed claims and report ambiguity.

Strict mode

`--strict` fails nonzero when:

- the diff cannot be fully evaluated;
- a blocking architectural finding exists;
- scope violation exists;
- required governed evidence is unavailable.

Non-strict mode always produces the report and uses disposition to communicate status.

15. Markdown design

The Markdown report should be compact.

Recommended format:

```
# Sensei Architectural Pre-Review

> **Disposition:** Evidence required
> **Coverage:** Governed
> **Risk:** High
> **Human decisions:** 2

## Executive summary

...

## Reviewer attention

1. **Is this a permanent writer?**
  ...

2. **Is runtime evidence mandatory?**
  ...

## Architectural impact

...

## Governance and scope

...

## Proof and evidence

...

## Result architecture

...

<details>
<summary>Applicable invariants and failure history</summary>
...
```

```
</details>

<details>
<summary>Bindings and receipt digests</summary>
...
</details>
```

Put high-value human questions near the top.

Put exhaustive provenance in collapsible sections or machine-readable output.

16. GitHub integration

Do not call GitHub from the core package.

The GitHub Action should:

1. check out full history;
2. install Sensei;
3. generate canonical JSON;
4. generate Markdown from that JSON;
5. upload JSON as an artifact;
6. write the job summary;
7. create or update one stable PR comment;
8. upload SARIF when findings have file locations.

Use a hidden marker:

```
<!-- sensei-architectural-pre-review-v1 -->
```

The action updates the existing comment instead of creating one on every push.

Delta update

On later PR revisions, show:

```
Resolved since previous report
New since previous report
Still open
Certification invalidated by new commit
Reviewer questions changed
```

The previous canonical report digest must be preserved.

17. Neural integration

Do not include neural output in the first implementation.

Later add a separate section:

```
neural_candidates:
  model_id:
  model_digest_sha256:
  dataset_digest_sha256:
  candidates:
    - prediction_id:
      task:
      proposition:
      calibrated_confidence:
      ood:
      supporting_refs:
      limitations:
      disposition: candidate
```

Allowed report language:

```
Candidate architectural anomaly
Likely boundary change
Possible ownership shift
Suggested architect question
```

Forbidden language:

```
Confirmed violation
Proven owner
Certified boundary
```

Neural candidates may contribute reviewer questions.

They may not change certification or terminal disposition.

18. Incremental implementation milestones

Milestone PR-0: contract spine

Goal:

Freeze the report before integrating live sources.

Deliver:

```
docs/design/architectural-pre-review-v1.md
schemas
fixtures
golang/architecture/prereview/model.go
vocabulary.go
normalize.go
validate.go
disposition.go
attention.go
render_json.go
render_markdown.go
tests
```

Use fixture-created typed inputs.

Do not add CLI, GitHub, neural code, ontology, or task-session changes.

Exit:

Sensei can canonicalize, validate, rank, and render a complete pre-review report from typed inputs without inventing architectural truth.

Milestone PR-1: advisory report

Integrate:

- Git diff binding;
- current edit-check/gate findings;
- briefings;
- impact;
- invariants;
- failure modes;
- forbidden fixes;
- required tests.

Add `sensei pre-review`.

Coverage:

advisory

Exit:

Any repository with a Sensei graph can generate a deterministic architectural pre-review before opening a PR.

Milestone PR-2: governed mutation report

After corrected Phase 3 is integrated, consume:

- actor;
- authority resolution;
- admission decision;
- capability consumption;
- observed change;
- scope verification.

Coverage:

governed

Exit:

The report can distinguish an ordinary diff from an authorized, single-use, scope-verified mutation.

Milestone PR-3: result architecture

After Phase 7, consume:

- result tree;
- result graph;
- artifact receipts;
- architectural delta;
- changed proof requirements.

Exit:

The report explains how the resulting architecture differs from the base architecture.

Milestone PR-4: proof and certification

After Phases 4–6 are rebased onto the live transaction, consume:

- evidence receipts;
- proof discharges;
- certification receipt.

Coverage:

```
proof_bound
```

Exit:

The report can display an independently verifiable correctness certification without recomputing or fabricating it.

Milestone PR-5: terminal report

After Phase 8, consume completion receipt.

Coverage:

```
terminal
```

Exit:

The report can distinguish certified work from terminal architectural closure.

Milestone PR-6: GitHub action

Deliver:

- PR comment;
- job summary;
- JSON artifact;
- SARIF;
- delta report.

Milestone PR-7: neural candidates

Add model candidates and architect-question ranking only after the neural importer is stable.

19. First implementation pull request

Claude should begin with Milestone PR-0 only.

Exact scope

```
docs/design/architectural-pre-review-v1.md

docs/schemas/architectural-pre-review/v1/
  report.schema.json
  reviewer-attention.schema.json

docs/fixtures/architectural-pre-review/v1/
  advisory-clean/
  governed-scope-verified/
  evidence-required/
  certified/
  cannot-verify/
  invalid/

golang/architecture/prereview/
  model.go
  vocabulary.go
  normalize.go
  validate.go
  identity.go
  disposition.go
  attention.go
  render_json.go
  render_yaml.go
  render_markdown.go
  *_test.go
```

Do not modify

```
golang/architecture/admission/**
golang/architecture/authority/**
golang/architecture/tasksession/**
golang/architecture/closure/**
golang/architecture/certification/**
golang/architecture/resultttransition/**
golang/architecture/ledger/**
cmd/awg/**
ontology/**
```



```
docs/awareness/**  
golang/server/embeddata/**
```

Required first-PR tests

```
TestPreReviewReportCanonicalRoundTrip  
TestPreReviewReportIDStable  
TestTemporaryPathsDoNotAffectDigest  
TestDispositionCannotClaimCertifiedWithoutReceipt  
TestDispositionCannotClaimTerminalWithoutCompletion  
TestScopeVerifiedDoesNotImplyCorrectness  
TestMissingEvidenceCannotPass  
TestReviewerAttentionRanksBlockingUnknownFirst  
TestReviewerAttentionDeduplicatesEquivalentQuestions  
TestModelCandidateCannotBecomeGovernedFinding  
TestMarkdownContainsReviewerAttentionBeforeDetail  
TestRepeatedRenderingByteIdentical  
TestUnknownVocabularyRejected  
TestCallerBooleanCannotForgeVerdict
```

First-PR definition of done

Sensei can represent, validate, canonicalize, rank, and render a complete architectural pre-review report from typed evidence without querying a repository, invoking an LLM, changing architectural behavior, or creating authority.

20. Security and privacy

1. Do not include source code bodies by default.
 2. Do not include secrets, tokens, credentials, or environment variables.
 3. Redact command stdout and stderr before embedding excerpts.
 4. Prefer artifact digests and paths over full content.
 5. Make private-repository report publication opt-in.
 6. Never put authentication or delegation receipts into public PR comments.
 7. Display principal IDs only when policy permits.
 8. Keep full receipt details in CI artifacts, with a summarized PR comment.
 9. Validate every path is repository-confined.
 10. Treat model-generated narrative as untrusted presentation.
-

21. Performance requirements

Target:

```
cached governed report: under 2 seconds  
advisory report: under 10 seconds for ordinary PRs
```

Expensive graph rebuilds and tests should be performed by the task workflow and consumed through receipts.

The report command should not rerun the full closure protocol unnecessarily.

Cache key:

```
report schema version  
+ base tree digest  
+ result tree digest  
+ graph digests  
+ ledger head  
+ policy digests  
+ selected output profile
```

22. Stop conditions

Stop rather than weakening the feature when:

- report code starts making admission decisions;
 - report code remaps proof receipts heuristically;
 - report code sets certification;
 - scope verification is labeled correct;
 - terminal closure is inferred without completion;
 - mutable projections become authoritative;
 - a narrative model introduces findings;
 - neural confidence changes a governed verdict;
 - GitHub metadata affects semantic identity;
 - report generation requires live runtime evidence when receipts already exist;
 - the implementation duplicates gate, closure, or certification logic;
 - a new RDF class is added without a real graph-query consumer.
-

23. Full definition of done

The feature is complete when:

1. A PR diff is deterministically bound.
 2. Architectural impact is explained.
 3. Applicable invariants and historical failure modes are shown.
 4. Typed authority and admission status are shown when available.
 5. Scope violations are explicit.
 6. Proof obligations and evidence status are shown.
 7. Result architecture changes are summarized.
 8. Epistemic states remain distinct.
 9. Reviewer attention is concise and actionable.
 10. Certification is displayed only from a valid receipt.
 11. Completion is displayed only from a valid receipt.
 12. Advisory mode works on repositories without a task ledger.
 13. Governed mode works from the verified ledger.
 14. Markdown and JSON are deterministic.
 15. GitHub comments update rather than multiply.
 16. Delta reports show new, resolved, and persistent findings.
 17. Neural candidates remain visibly non-authoritative.
 18. No report path can authorize, certify, or complete a task.
-

24. Expected Claude completion report

Commits:

- <hash> docs(review): freeze architectural pre-review v1
- <hash> feat(review): add canonical pre-review report model
- <hash> feat(review): rank deterministic reviewer attention
- <hash> feat(review): render deterministic markdown and JSON

Contract:

- schema version:
- report dispositions:
- coverage levels:
- epistemic states:
- fixture digest:

Canonical identity:

- report ID:
- semantic digest:
- temporary-path test:
- repeated-render test:

Reviewer attention:

- ranking policy:
- maximum default items:
- deduplication:
- blocking-first proof:

Safety:

- certification forgery refused:
- terminal forgery refused:
- scope/correctness separation:
- neural authority leakage refused:
- caller-boolean forgery refused:

Fixtures:

- positive:
- invalid:
- all validated:

Verification:

- prereview tests:
- architecture tests:
- build:
- check:
- validate:
- git diff --check:

Modified existing architectural behavior:

- false

Next legal task:

- Milestone PR-1 advisory report integration